

Propuesta de solución al problema de concurrencia en la comunicación SPI de la DAQ basada en dsPIC

Haniel Ulises
Servicio social

2 de octubre de 2026

Resumen

El sistema DAQ-SPI emula una tarjeta de adquisición de datos tipo Quanser mediante un microcontrolador dsPIC33CK64MC105, un puente USB-SPI FT2232H y un conjunto de S-Functions de Simulink. Se ha observado que la comunicación se corrompe o se interrumpe cuando la lectura del encoder y la escritura al DAC operan simultáneamente. En este documento se analiza el código existente, se identifican las causas del problema y se propone un esquema de comunicación basado en una única transferencia SPI full-duplex de longitud fija por paso de simulación, delimitada por la señal de selección de esclavo y atendida en el microcontrolador mediante DMA.

1. Descripción del sistema

La arquitectura actual se muestra en la figura 1. En la PC, un modelo de Simulink utiliza tres S-Functions escritas en C++ sobre la biblioteca libMPSSE-SPI de FTDI. La función `initializePic` abre el canal SPI del FT2232H y comparte su *handle*, protegido por un mutex del sistema operativo, con las otras dos: `escribirPic`, que convierte un voltaje en el intervalo $[-2.5, 2.5]$ V a un valor de 16 bits, y `leerPic`, que obtiene la posición del encoder.

El FT2232H opera como maestro SPI a 1 MHz en modo 0. El dsPIC actúa como esclavo en su módulo SPI1, como maestro del DAC en el módulo SPI2, y cuenta los pulsos del encoder con el módulo QE11. El reloj del microcontrolador es el oscilador interno FRC de 8 MHz sin PLL, lo que da una frecuencia de instrucción $F_{cy} = 4$ MHz.

La escritura y la lectura se realizan como transacciones independientes. Para escribir, la PC ejecuta `SPI_Write` con tres bytes: el encabezado `0xAA` y el valor del DAC en dos bytes. Para leer, ejecuta `SPI_Read` de doce bytes y busca en ellos la secuencia `0xAA 0x55` seguida de los cuatro bytes de la posición.

En el dsPIC, el firmware (`dsPicController`) consiste en un lazo infinito que alterna dos funciones. `escrituraDac()` toma los bytes disponibles en la FIFO de recepción de SPI1, reconstruye el valor mediante una máquina de estados que busca el encabezado `0xAA` y lo transmite al DAC por SPI2. `lecturaEncoder()` escribe seis bytes en la FIFO de transmisión de SPI1 cada vez que ésta no está llena.

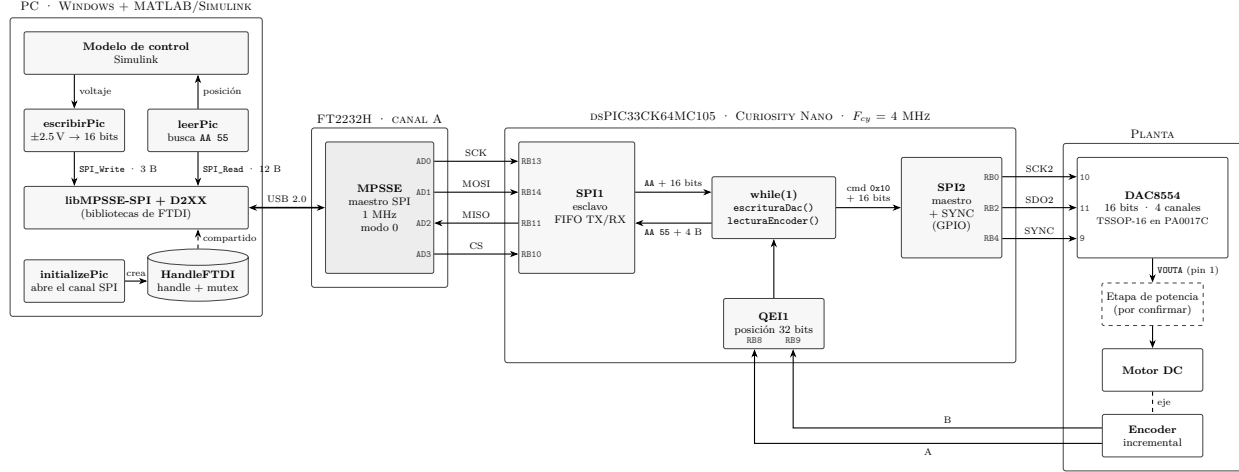


Figura 1: Arquitectura actual del sistema.

2. Análisis del problema

2.1. Uso del bus como canal half-duplex

En SPI cada pulso de reloj desplaza simultáneamente un bit en cada dirección, de modo que toda transferencia es necesariamente de lectura y escritura a la vez. El protocolo actual no aprovecha esta propiedad. Durante un `SPI_Write`, el dsPIC entrega por MISO bytes de su FIFO de transmisión que la PC descarta; durante un `SPI_Read`, la PC envía bytes de relleno que ingresan a la FIFO de recepción del dsPIC como si fueran datos. Ambas transacciones consumen o introducen bytes que el otro extremo no espera, y el flujo de datos pierde la alineación.

2.2. Gestión de las FIFOs en el esclavo

De acuerdo con la hoja de datos del dsPIC33CK64MC105, la FIFO del módulo SPI tiene una profundidad de cuatro elementos en modo de 8 bits. `lecturaEncoder()` intenta escribir seis bytes, por lo que `SPI1_ByteWrite()` permanece en espera activa hasta que el maestro genere pulsos de reloj suficientes para liberar espacio. Dado que el esclavo no controla el reloj, el tiempo de esa espera depende por completo de la PC.

Mientras el lazo está detenido en esa espera, `escrituraDac()` no se ejecuta y la FIFO de recepción, también de cuatro elementos, puede llenarse. En ese caso el módulo activa la bandera de desbordamiento `SPIROV` y deja de aceptar datos. El controlador generado por MCC no limpia esa bandera, por lo que la recepción queda suspendida de forma permanente. Las llamadas a `printf` dentro del lazo aumentan el tiempo entre lecturas y agravan la situación.

Adicionalmente, como la FIFO de transmisión se llena sin relación con el momento en que la PC lee, los valores de posición que recibe la PC tienen una antigüedad indeterminada y no están alineados con el inicio de la transacción. Por esta razón `leerPic` necesita buscar el encabezado dentro de una ventana de doce bytes.

2.3. Delimitación de tramas

El dsPIC no utiliza la señal de selección de esclavo para identificar el inicio y el fin de una trama; se limita a buscar el byte `0xAA` en el flujo recibido. Este valor puede aparecer también dentro de los

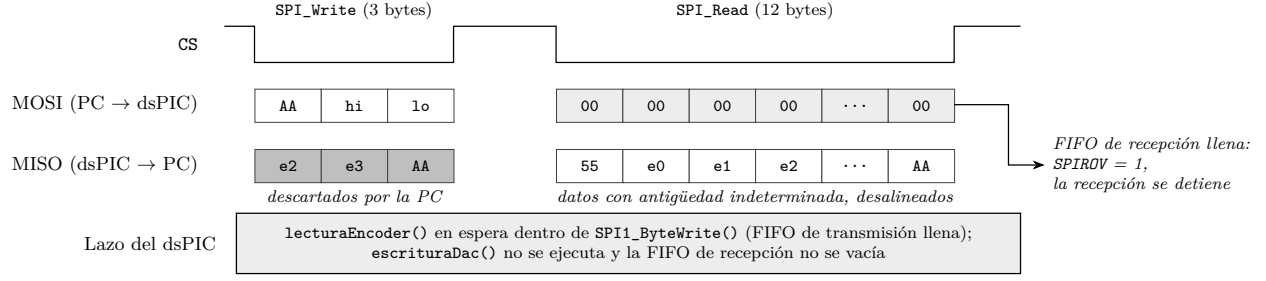


Figura 2: Comportamiento del protocolo actual. Los bytes sombreados en MISO son descartados por la PC; los bytes de relleno en MOSI ingresan a la FIFO de recepción del dsPIC.

datos, y una vez perdida la alineación no existe un mecanismo que la restablezca.

2.4. Observaciones adicionales

El mutex implementado en la PC impide que dos S-Functions accedan al FT2232H al mismo tiempo, pero no interviene en ninguno de los fenómenos anteriores, que ocurren en el uso del bus y en el firmware del esclavo. Por otra parte, al terminar la simulación `initializePic` envía el valor `0x0000`, que con la conversión empleada corresponde a -2.5 V y no a 0 V; el motor queda energizado después de detener el modelo.

3. Solución propuesta

La propuesta consiste en realizar una sola transferencia full-duplex de longitud fija en cada paso de simulación, mediante la función `SPI_ReadWrite` de `libMPSSE`. En esa transferencia la PC envía el valor del DAC y, simultáneamente, recibe la posición del encoder. La señal de selección de esclavo delimita cada trama: el dsPIC procesa la trama recibida cuando la señal regresa a nivel alto y prepara la respuesta siguiente antes de la próxima transferencia. El movimiento de datos entre la memoria y el módulo SPI1 lo realiza el controlador DMA, de manera que el programa principal nunca espera al maestro. Este esquema corresponde al de una tarjeta de adquisición comercial, en la que cada paso de simulación ejecuta una lectura y una escritura de forma atómica.

3.1. Formato de trama

Ambas direcciones utilizan tramas de ocho bytes, descritas en la tabla 1. El número de secuencia y el código de redundancia cíclica permiten a la PC detectar tramas corruptas o perdidas y descartarlas en lugar de interpretarlas como datos válidos. Se conserva la conversión actual del voltaje, valor = $13107(V + 2.5)$, de modo que el valor `32767` corresponde a 0 V.

El byte `flags` contiene, en su bit 0, la habilitación de la salida analógica (con valor cero el DAC se lleva a 0 V) y, en su bit 1, la orden de poner en cero el contador del encoder. El byte `status` indica en sus bits 0, 1 y 2, respectivamente, si la última trama recibida tuvo error de CRC, si tuvo un encabezado inválido y si se activó el temporizador de vigilancia.

3.2. Temporización

La figura 3 muestra la secuencia en un periodo de muestreo. A 1 MHz, la transferencia de ocho bytes dura aproximadamente $64 \mu\text{s}$. La respuesta enviada en la transferencia k contiene la

Byte	PC → dsPIC (MOSI)		dsPIC → PC (MISO)	
0	0xA5	inicio de trama	0x5A	inicio de trama
1	seq	número de secuencia	seq	eco de la última trama válida
2	dac_h	valor del DAC, byte alto	enc0	posición del encoder
3	dac_l	valor del DAC, byte bajo	enc1	(entero de 32 bits
4	flags	banderas de control	enc2	con signo,
5	—	reservado	enc3	<i>little-endian</i>)
6	—	reservado	status	estado del dsPIC
7	crc	CRC-8 de los bytes 0 a 6	crc	CRC-8 de los bytes 0 a 6

Tabla 1: Formato de trama propuesto. El CRC-8 utiliza el polinomio 0x07.

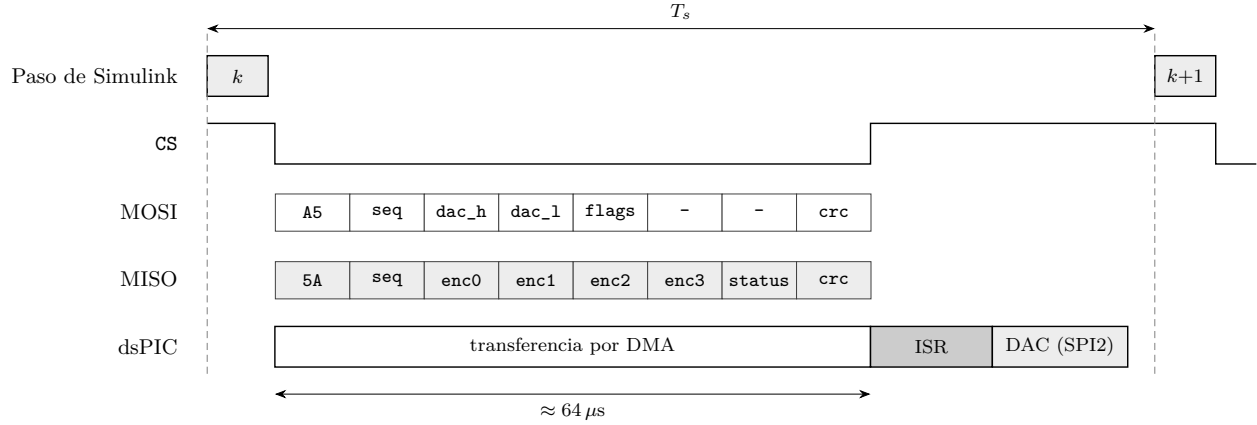


Figura 3: Secuencia en un periodo de muestreo. La interrupción asociada al flanco de subida de CS procesa la trama y prepara la respuesta antes de la siguiente transferencia.

posición muestreada al término de la transferencia $k - 1$, por lo que el dato tiene, a lo sumo, un periodo de antigüedad. Este retardo es conocido y constante, a diferencia del esquema actual, y debe considerarse en el diseño del controlador.

3.3. Firmware del dsPIC

La organización propuesta del firmware se muestra en la figura 4. Dos canales del controlador DMA atienden el módulo SPI1: uno transfiere el buffer de respuesta `tx_buf` hacia el registro de transmisión y otro copia los datos recibidos en `rx_buf`. El flanco de subida de la señal de selección, conectada a RB10, genera una interrupción por cambio de nivel.

En la interrupción de fin de trama se verifica el encabezado y el CRC de `rx_buf`; si son correctos, se almacenan el valor del DAC y las banderas y se actualiza el número de secuencia, y en caso contrario se registra el error en `status`. A continuación se deshabilita y se vuelve a habilitar el módulo SPI1. Según la hoja de datos, la deshabilitación reinicia el módulo, con lo que se vacían las FIFOs y se borra cualquier condición de desbordamiento; así cada trama comienza alineada aunque la anterior haya fallado. Después se lee el contador del QEI, se construye en `tx_buf` la respuesta con su CRC y se reprograman ambos canales de DMA. Finalmente se reinicia el temporizador de vigilancia y se indica al programa principal que hay un valor nuevo para el DAC.

El programa principal se limita a transmitir al DAC, por SPI2, los valores nuevos que reporta la interrupción. Al tratarse de un periférico distinto, esta operación no interfiere con la comunicación con la PC. El temporizador de vigilancia lleva el DAC a 0 V si transcurren 50 ms sin recibir una

independiente de Simulink, que envíe al menos cien mil tramas con valores conocidos y contabilice los errores de CRC y de número de secuencia; el criterio de aceptación es la ausencia de errores. Durante estas dos etapas se utilizará un analizador lógico sobre las señales SCK, MOSI, MISO y CS para verificar la forma de las tramas y medir la duración de la interrupción. En la tercera etapa se implementará el nuevo bloque de Simulink, y en la cuarta se realizarán pruebas con el motor: una rampa de voltaje verificada con osciloscopio y la comparación de la posición medida contra un desplazamiento conocido. En esta etapa se medirá también el tiempo de ida y vuelta por USB para determinar el periodo de muestreo mínimo alcanzable.

5. Consideraciones

El cambio de protocolo afecta tanto al firmware como a las S-Functions, por lo que ambas partes deben actualizarse de manera conjunta. La latencia del bus USB y la planificación de procesos de Windows limitan el periodo de muestreo mínimo y pueden introducir variaciones entre pasos, con independencia del firmware. Por último, la configuración del DMA en modo esclavo es la parte más delicada de la implementación; por ello se validará con el programa de prueba antes de integrarla con Simulink.

Referencias

- [1] Microchip Technology, *dsPIC33CK64MC105 Family Data Sheet*, DS70005399D.
- [2] Microchip Technology, *dsPIC33CK64MC105 Curiosity Nano User's Guide*, DS70005517.
- [3] Future Technology Devices International, *AN_178: User Guide for libMPSSE-SPI*.